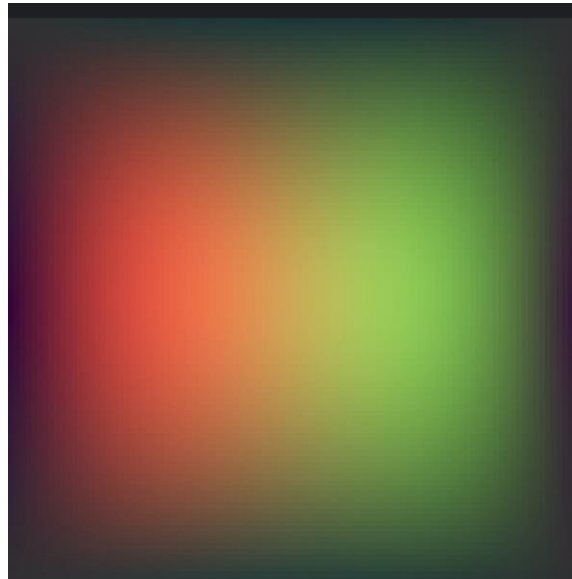
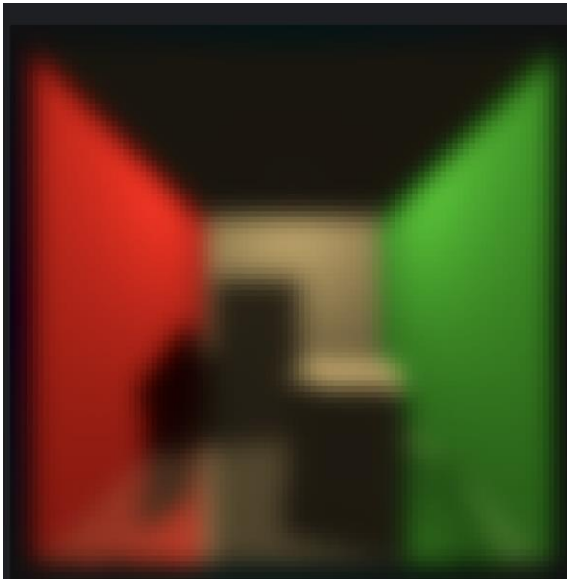
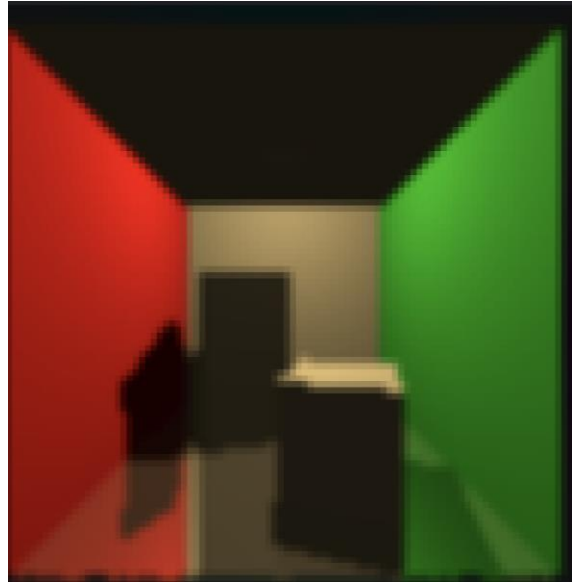
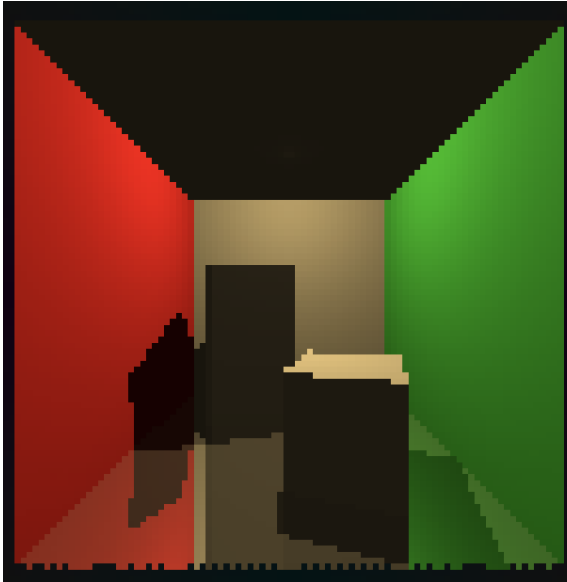


Post Processing

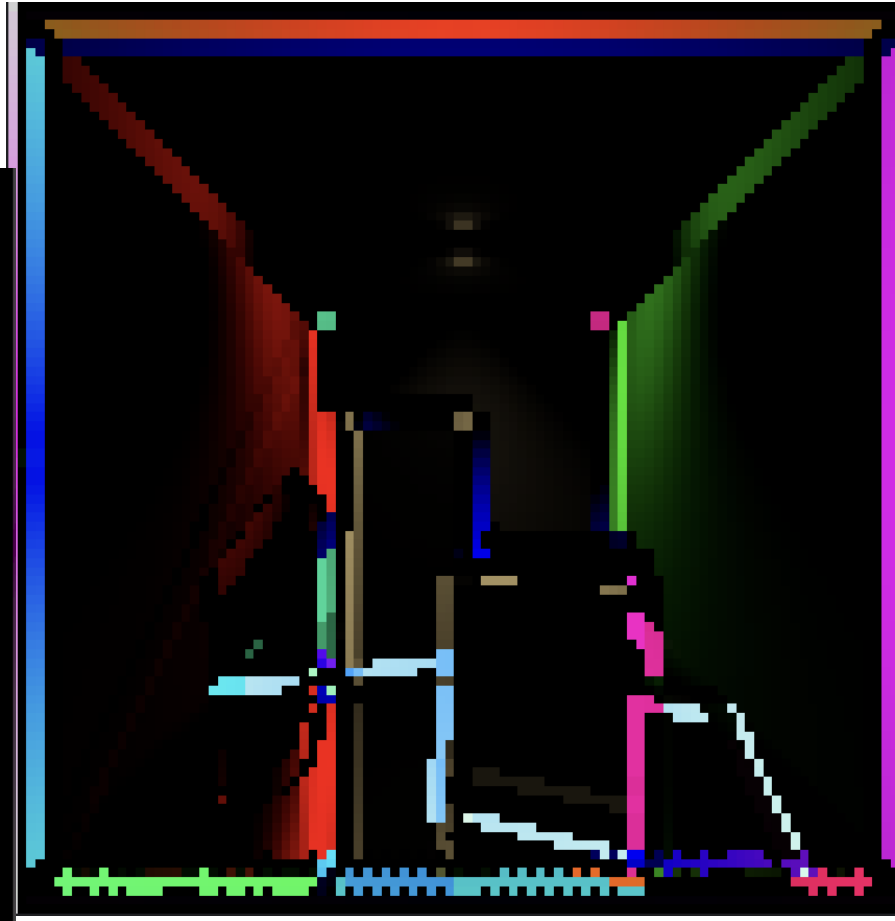
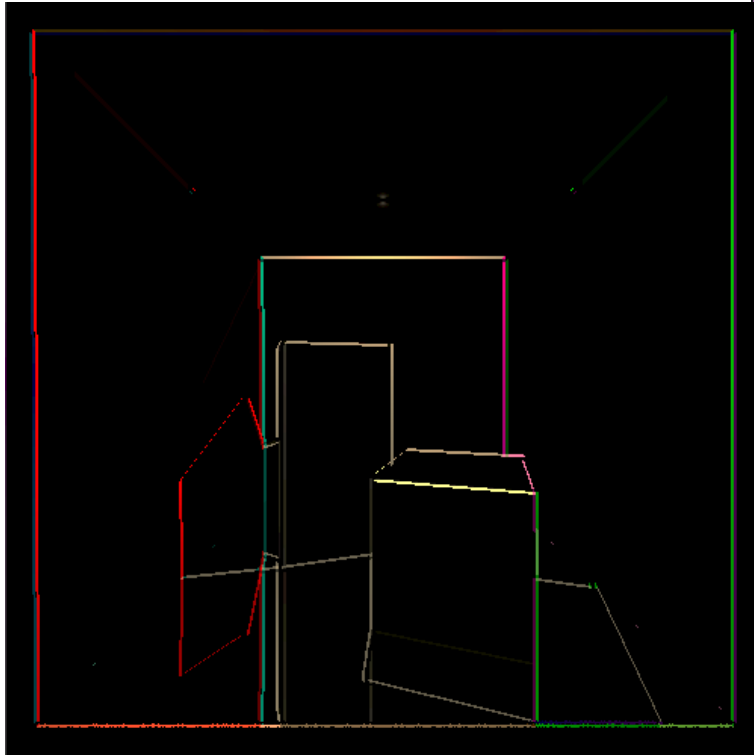
Atli Arnarsson, Henry Morin, Andrew Petersen, Drake Ma



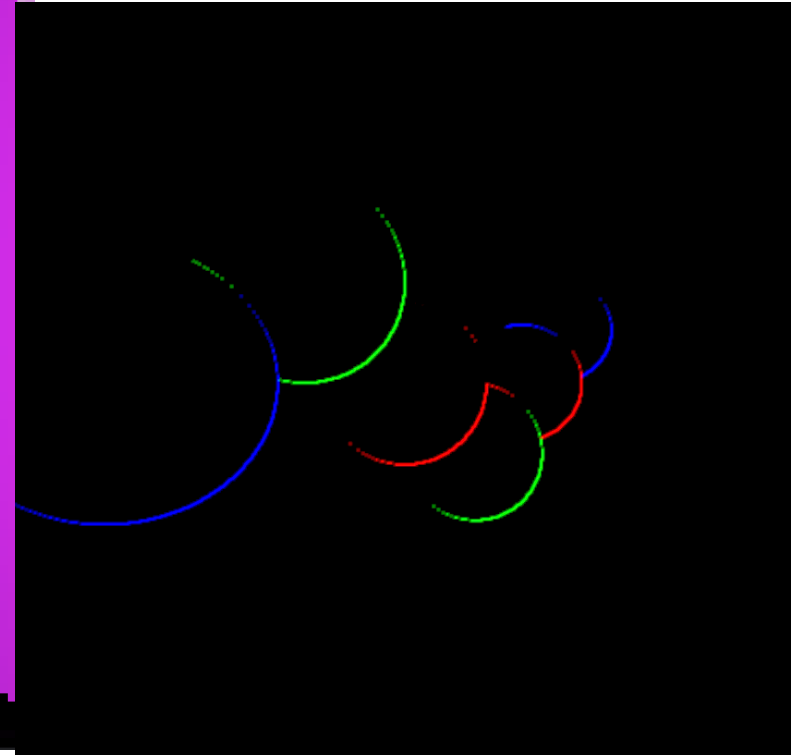
Blur

- Separable convolution
- 3x3 Gaussian blur kernel
- 0, 1, 10, 1000 passes

Edge Detect

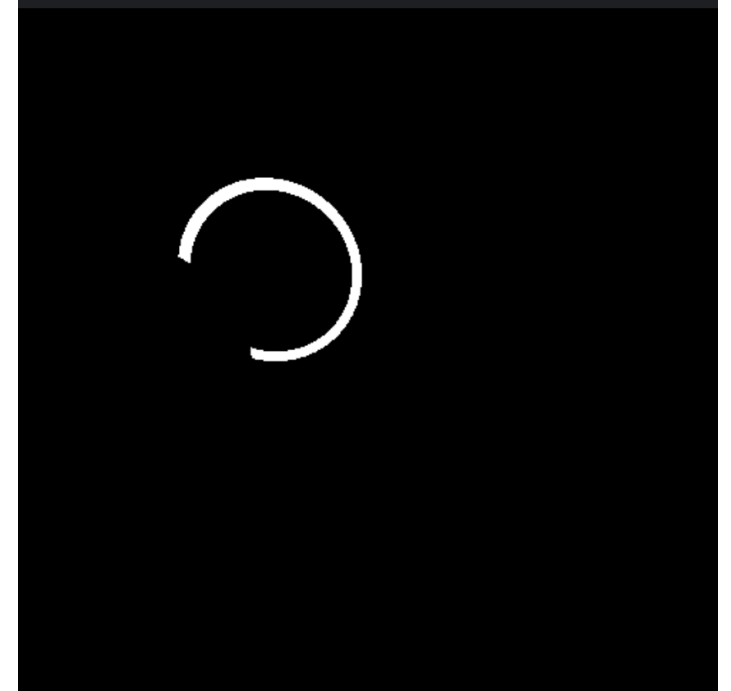
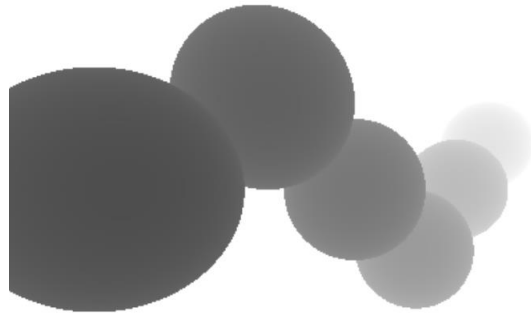


- Sobel Filter
- Two convolution kernels, X & Y



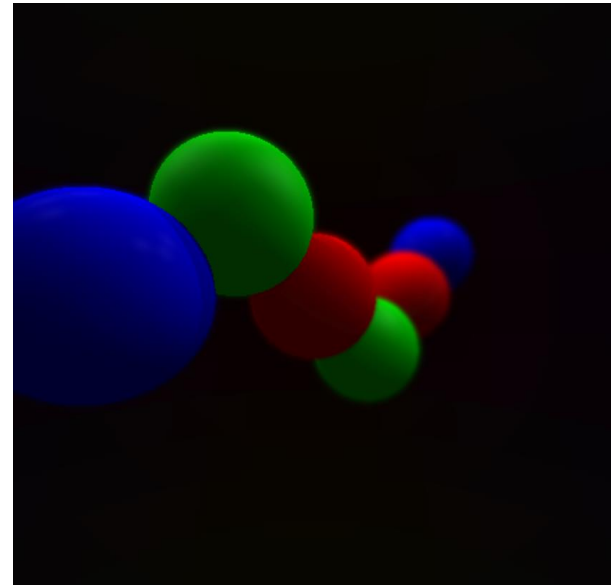
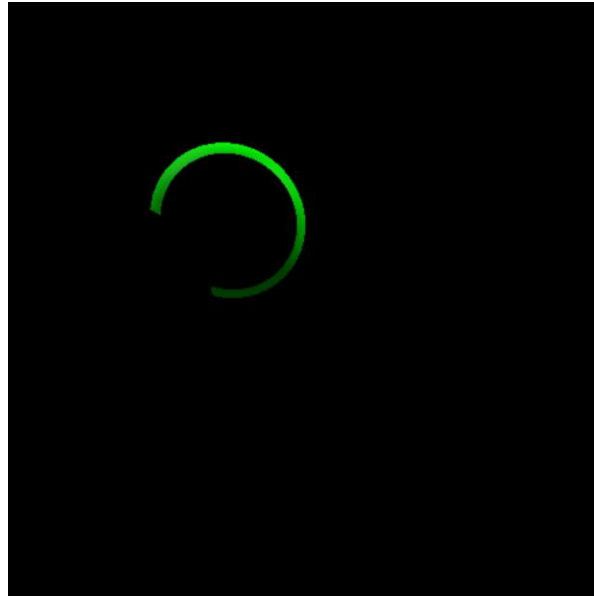
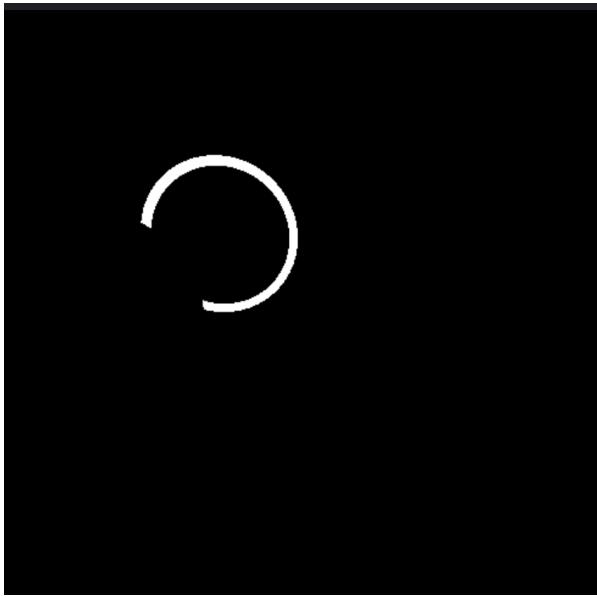
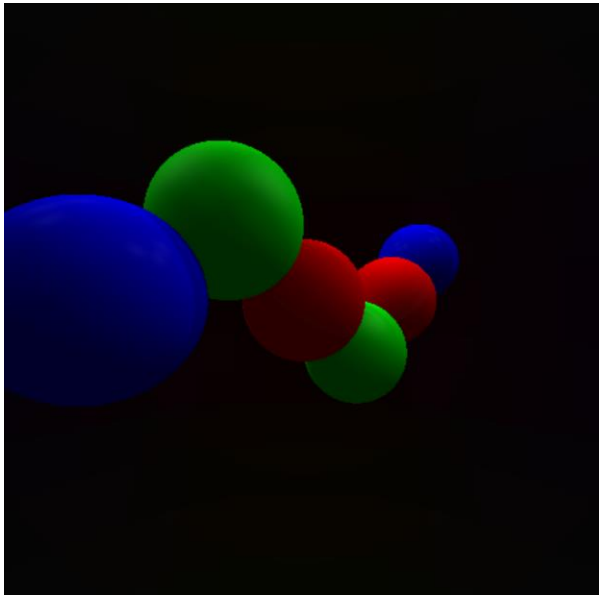
Threshold

- Pixels within a certain range turn white
- Pixels outside the range turn black
- Used with depth map to create mask over the pixels a certain distance from the camera

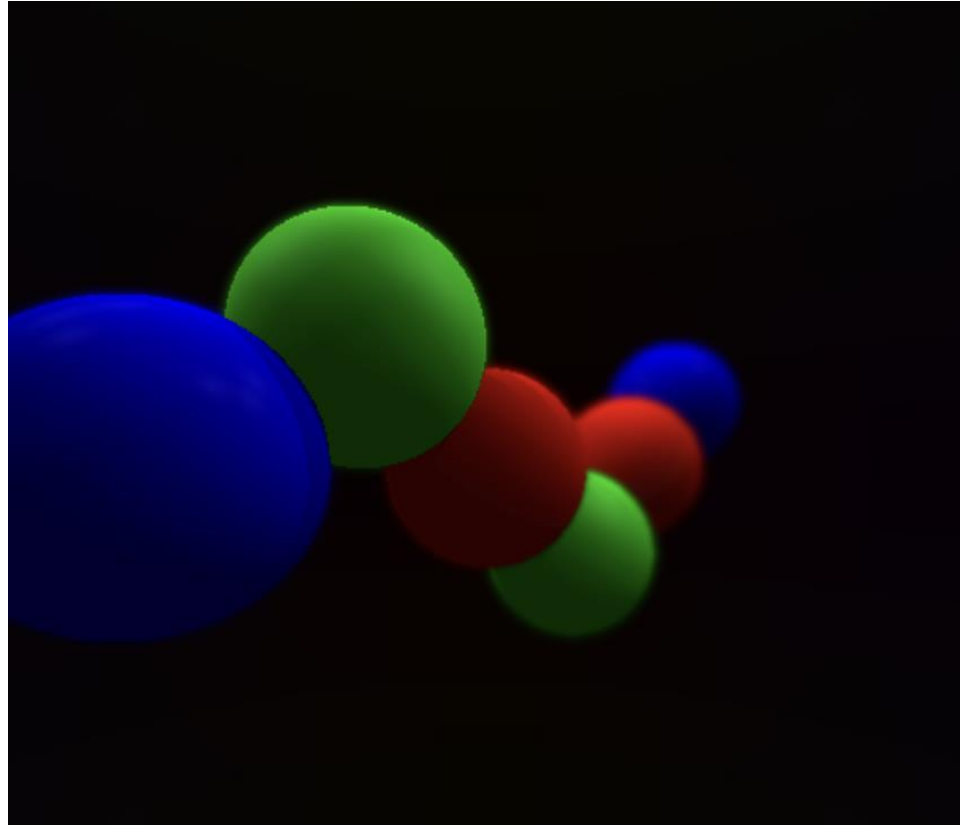


Depth of Field

- Iterate over all depths of the scene and progressively blur
- Used effect pipeline features to build depth of field using other effects



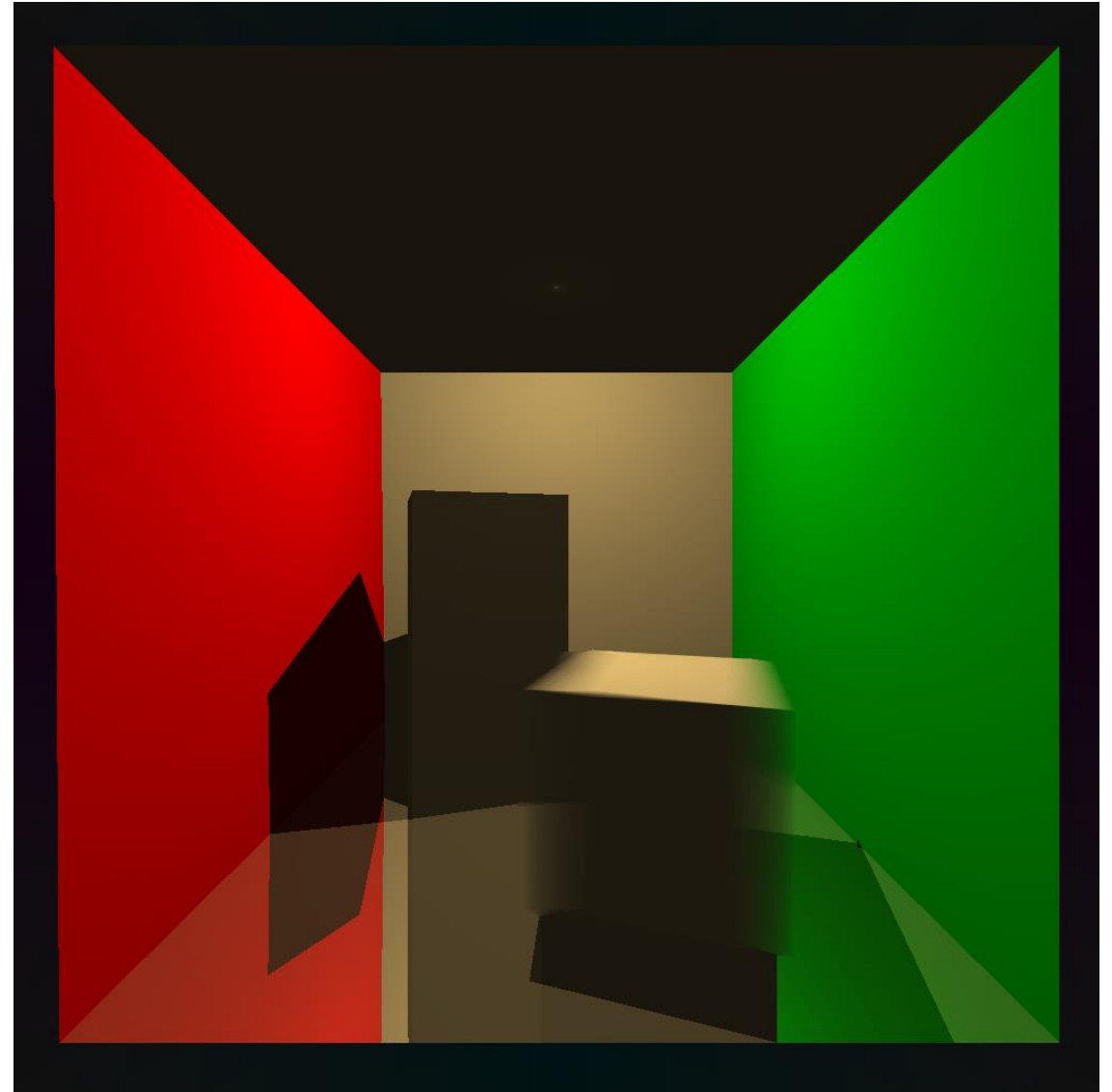
Depth of Field Gallery



Motion Blur

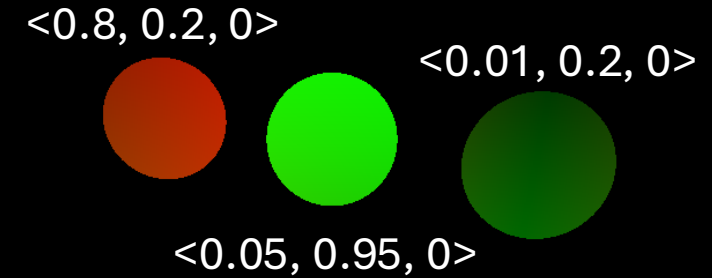
Goal:

Apply an area of directional blur to objects in the scene as if they're in motion.



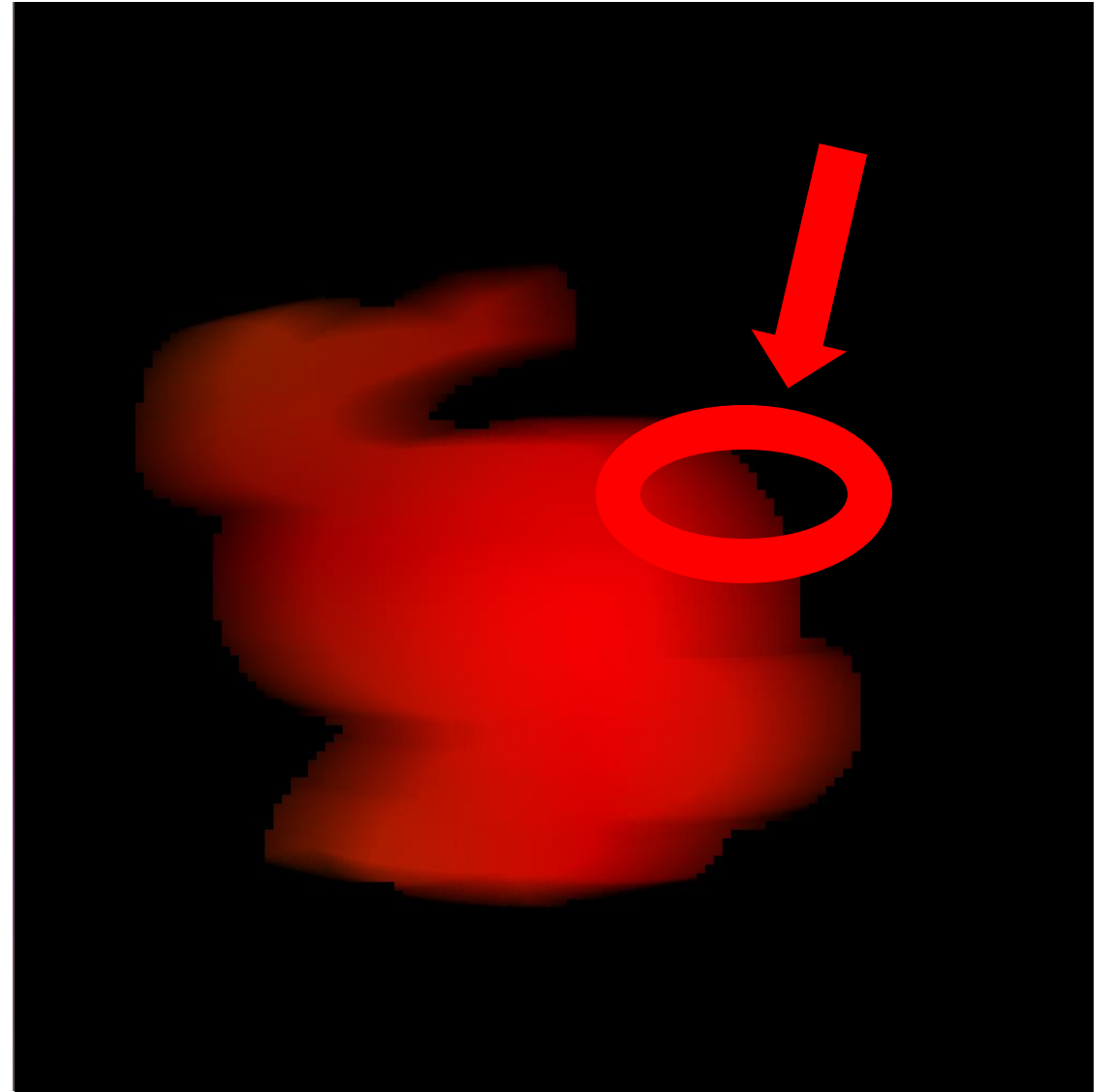
How do we know if something is moving?

- Each shape would contain a velocity (`Vector3`)
- When ray hits shape, `Hitpoint` receives velocity
- Project velocity in the direction of ray so that `x` and `y` are in terms of camera's 2D space
- Scale velocity with focal length and depth



Apply velocity to image?

- We might be missing a step...
- Only pixels with velocity were blurred
- Need to blur the velocity buffer as well

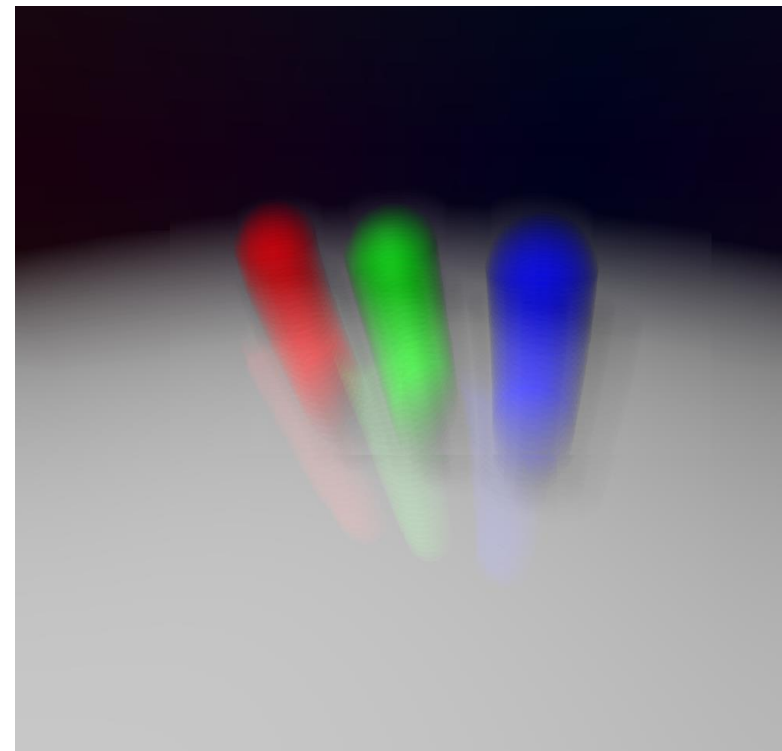
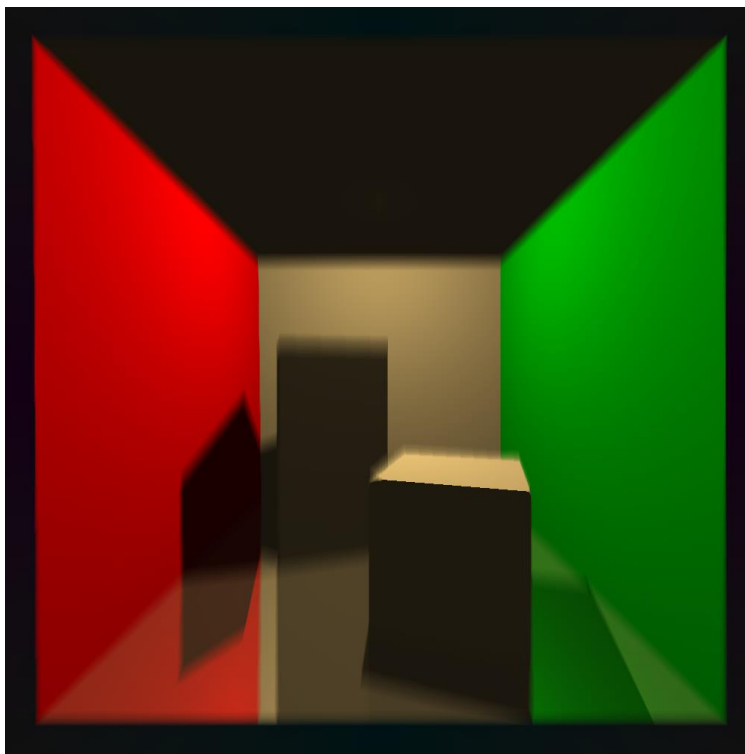
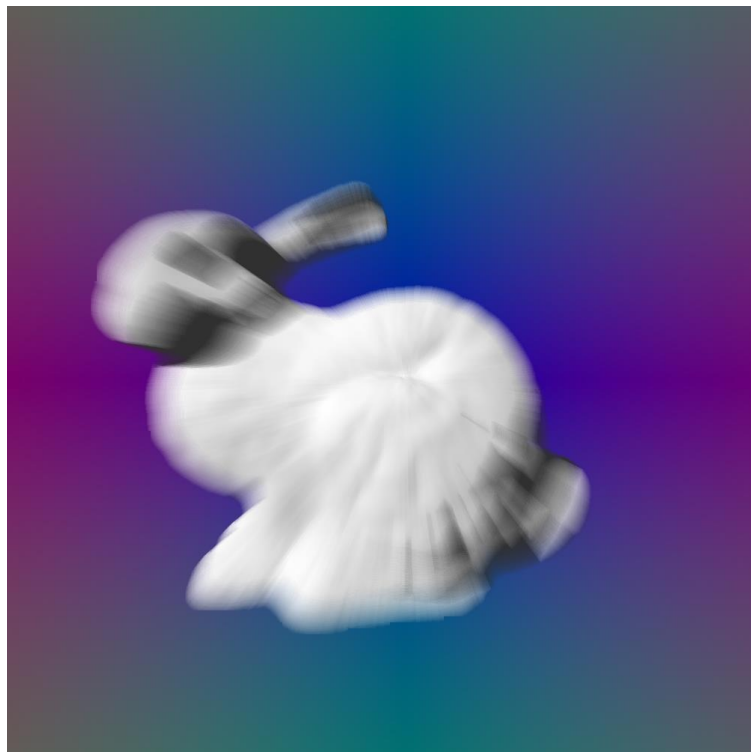


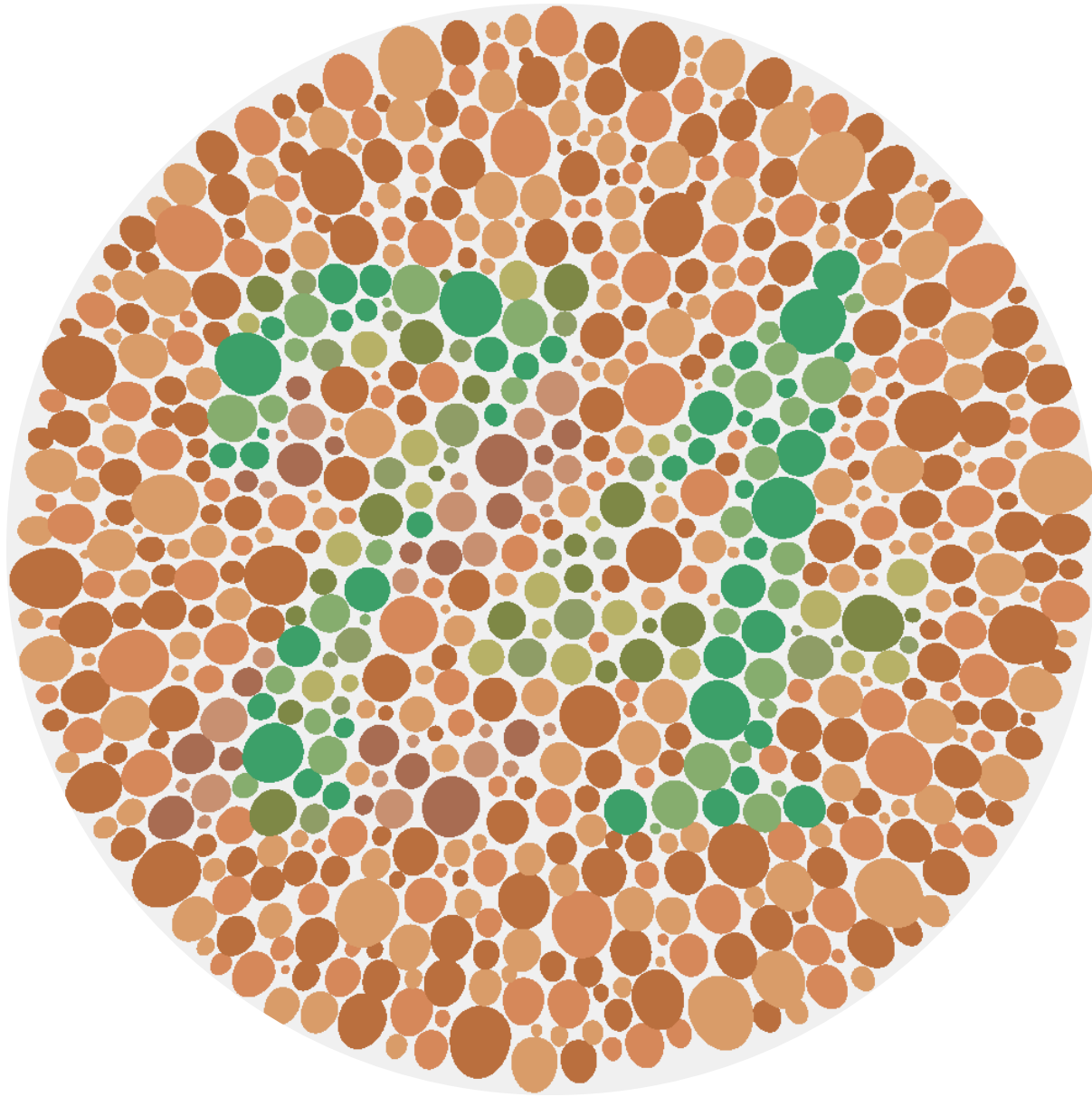
Blurring the velocity buffer

1. Divide the velocity buffer into neighbors of chunks, each neighbor contains max velocity from its pixels
2. Get the max velocity from neighboring chunks
3. Use the max velocity to blur the velocity buffer
4. Use the new velocity buffer and blur the image!



Motion Blur Gallery





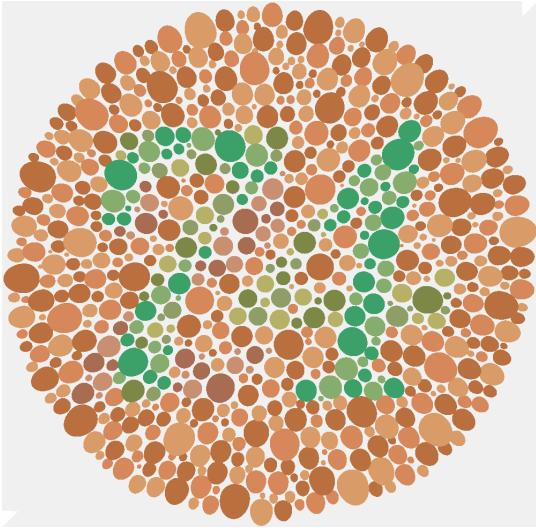
Daltonization

Goal:

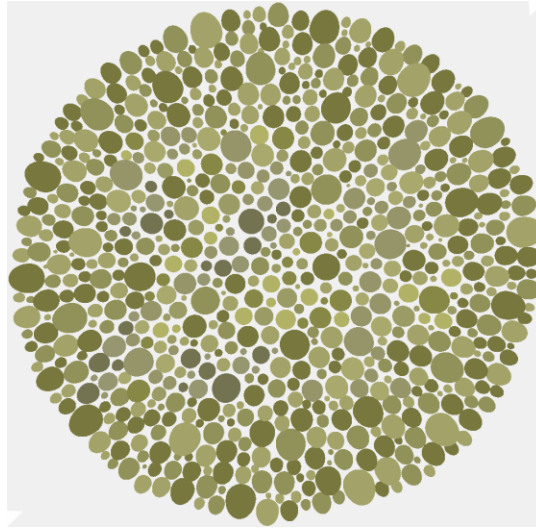
Reduce color confusion for people with Color Vision Deficiency (CVD) by enhancing the color they can see and reduce the color they can't see

Color Vision Deficiency (CVD)

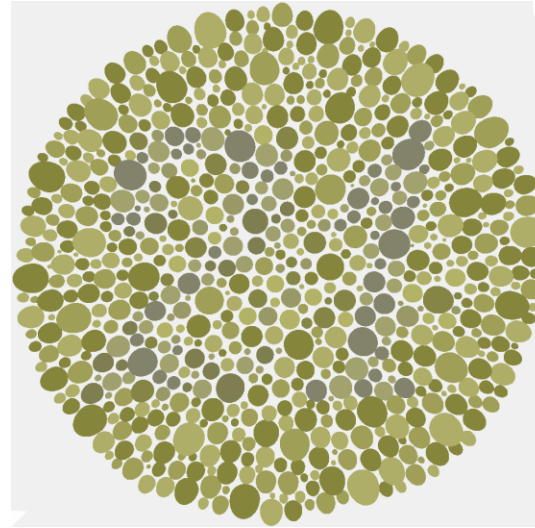
Normal Vision



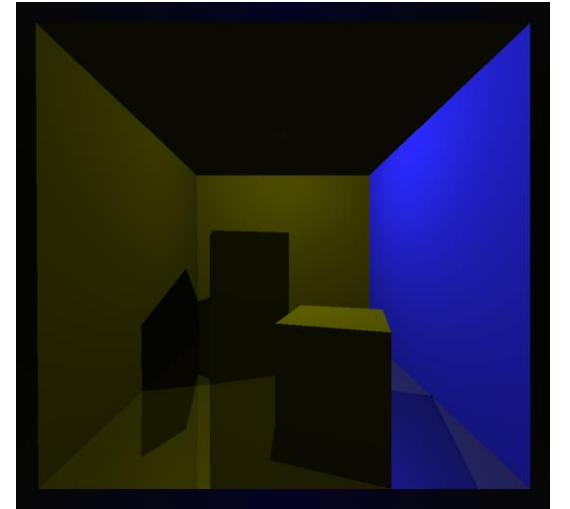
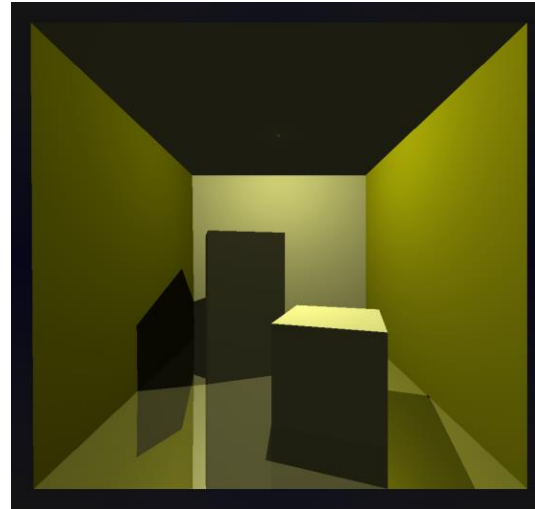
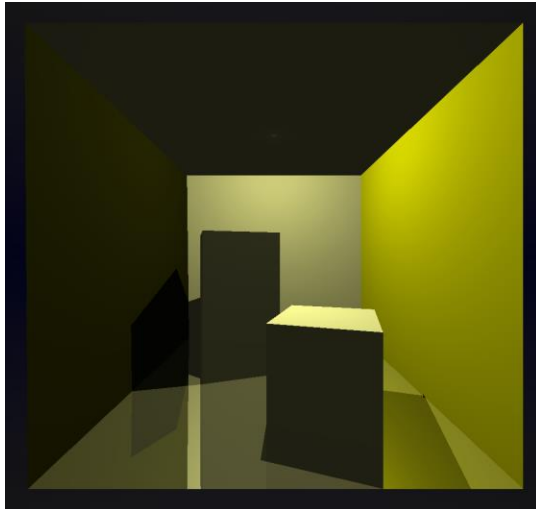
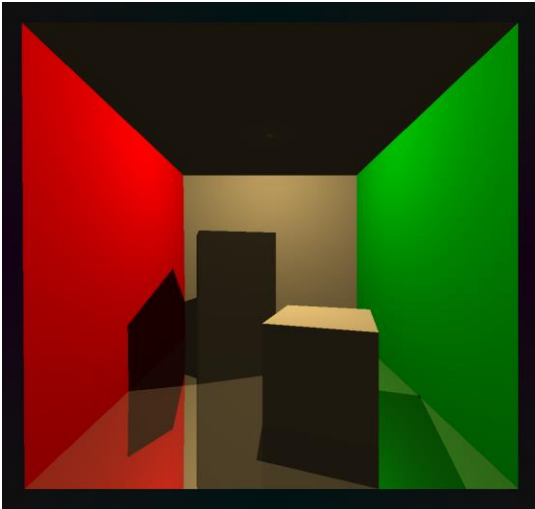
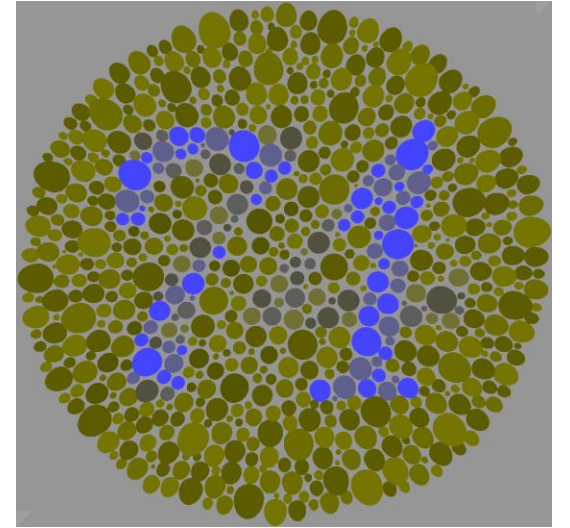
Simulated Protanopia

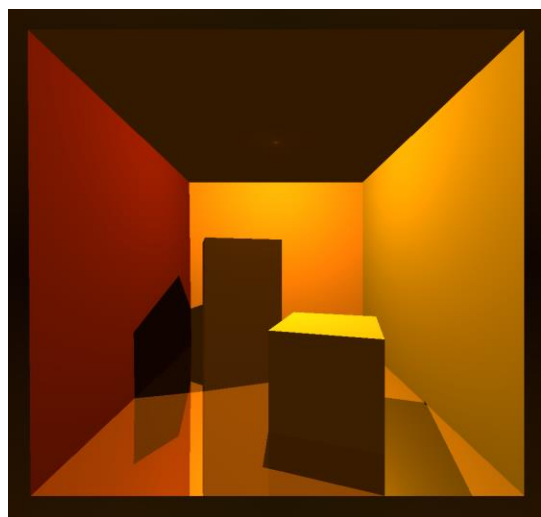


Simulated Deuteranopia

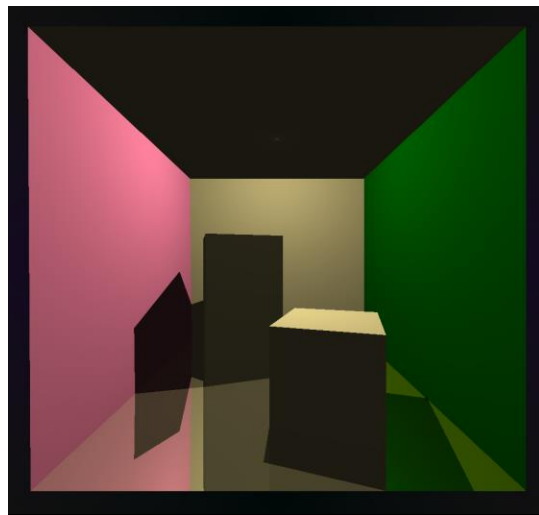


Simulated Tritanopia

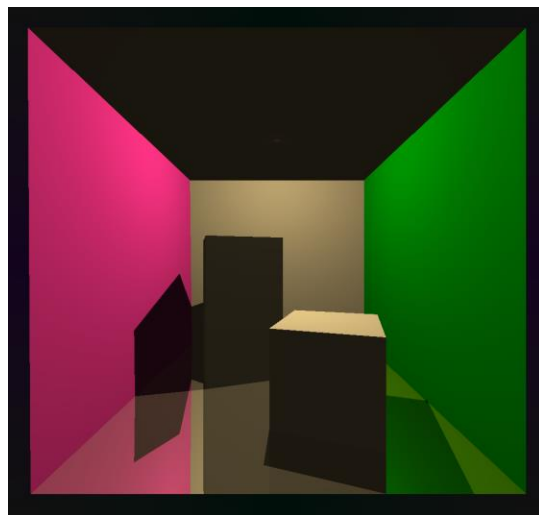




Fix for Protanopia



Fix for Deuteranopia

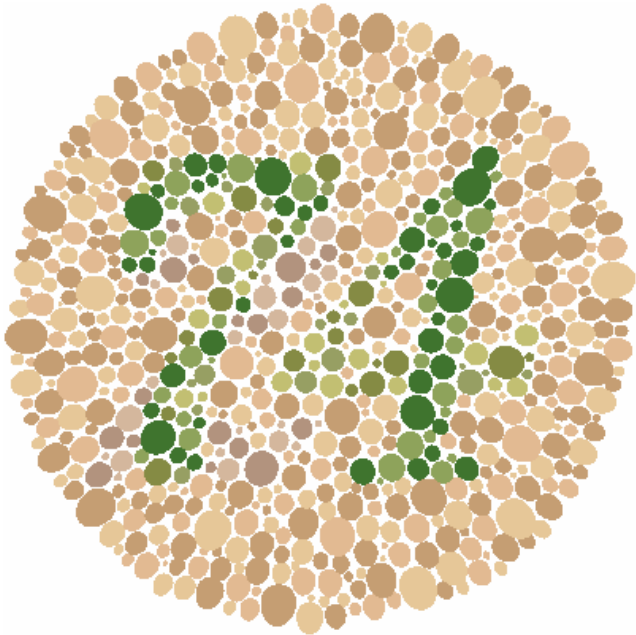


How do we compensate for the missing colors?

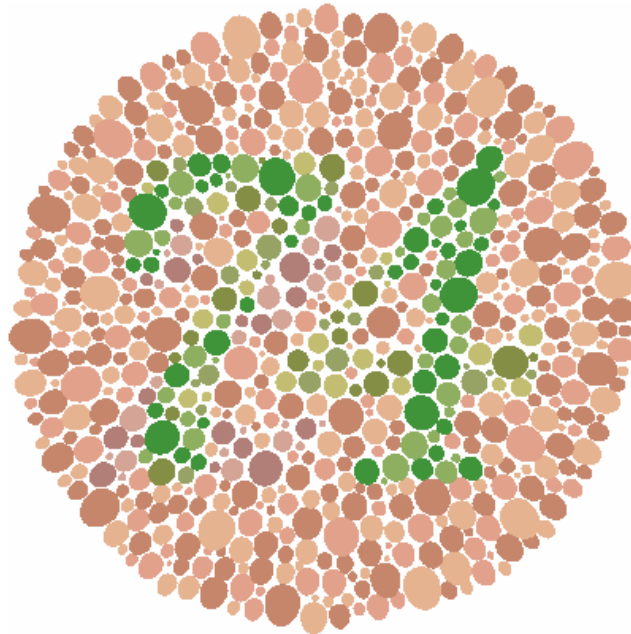
1. Convert RGB to LMS (Long Medium Short)
2. Simulate what people with the CVD sees
3. Convert LMS back to RGB
4. Subtract CVD image from original image to get the "error" as the missing colors
5. Calculate the additional color to add to the final image based off the missing colors
6. Add it to the original image to fix it!

Daltonization Gallery

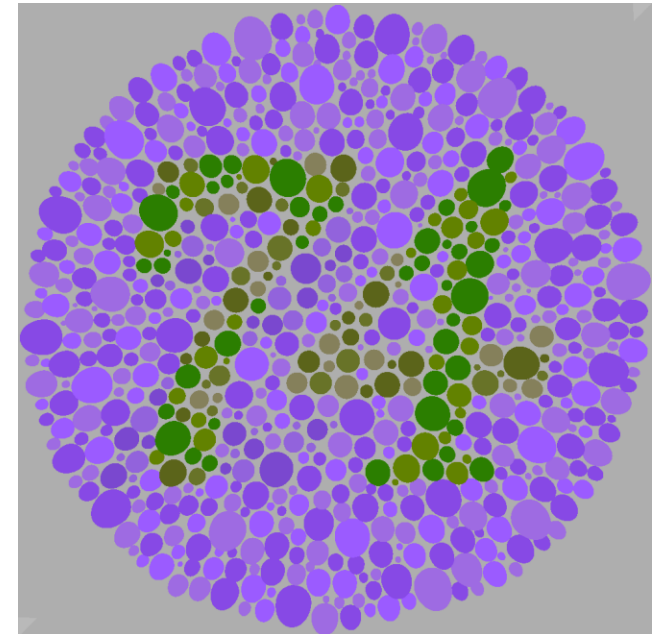
Fixed for Protanopia



Fixed for Deuteranopia



Fixed for Tritanopia



Vignette

Goal: Replicate effect found in photography where the borders of the frame are darker than the center.

Algorithm:

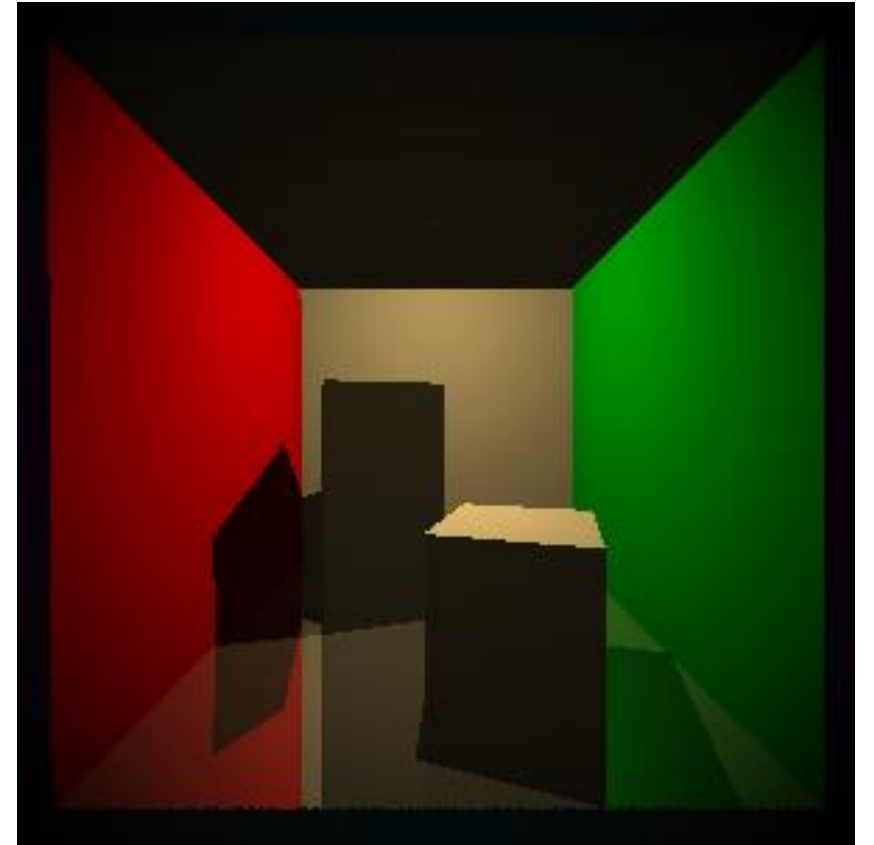
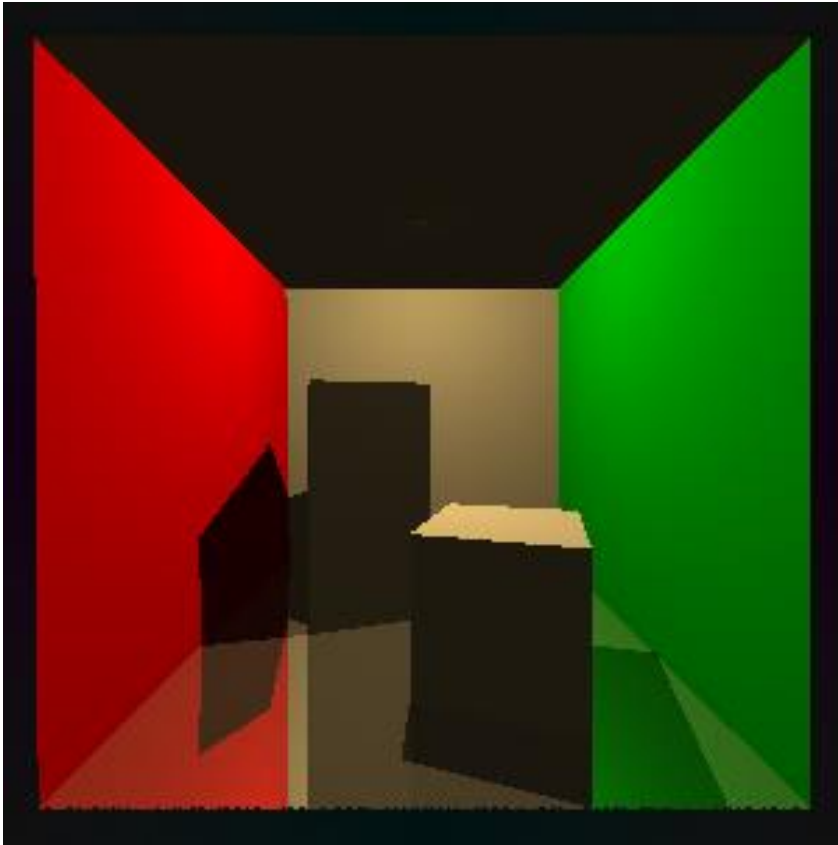
For x in image; for y in image do

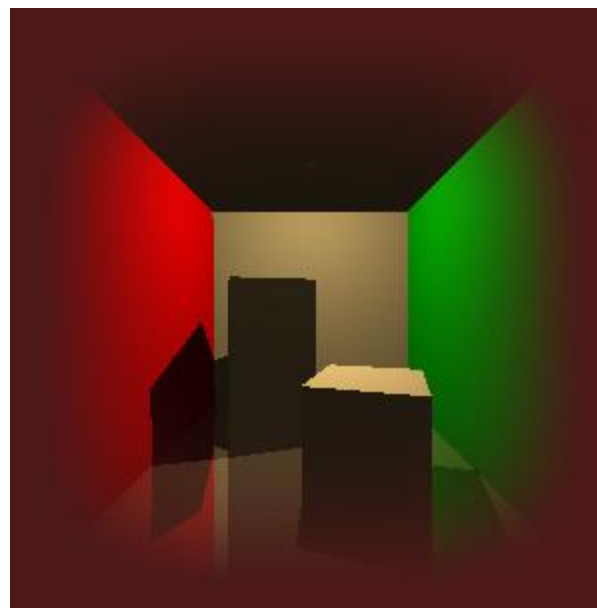
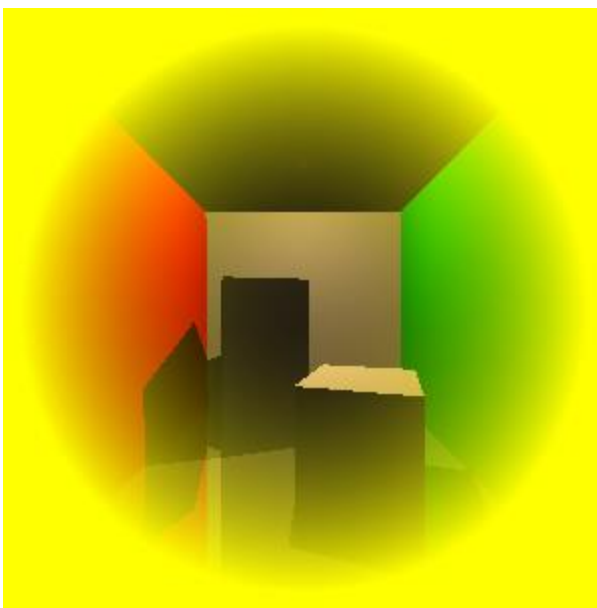
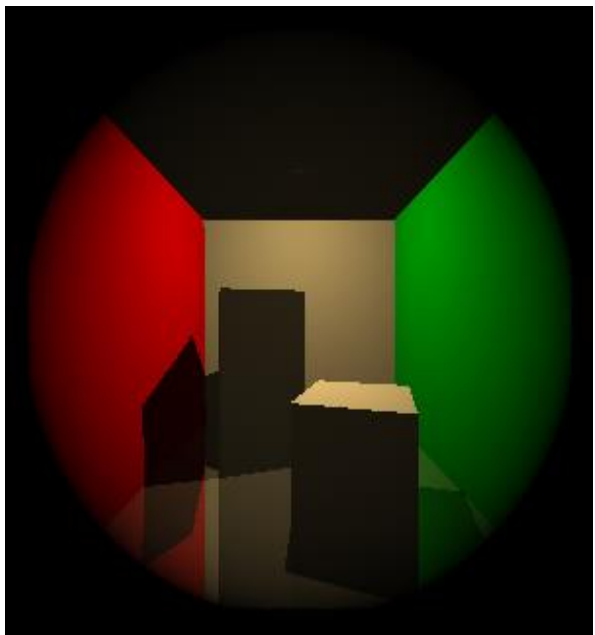
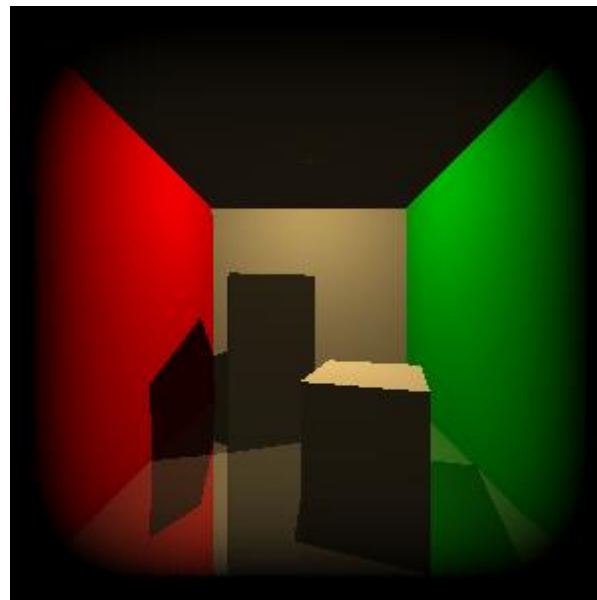
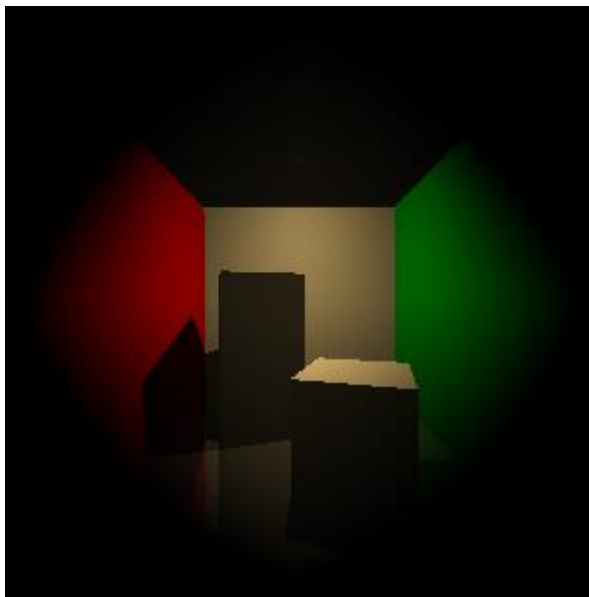
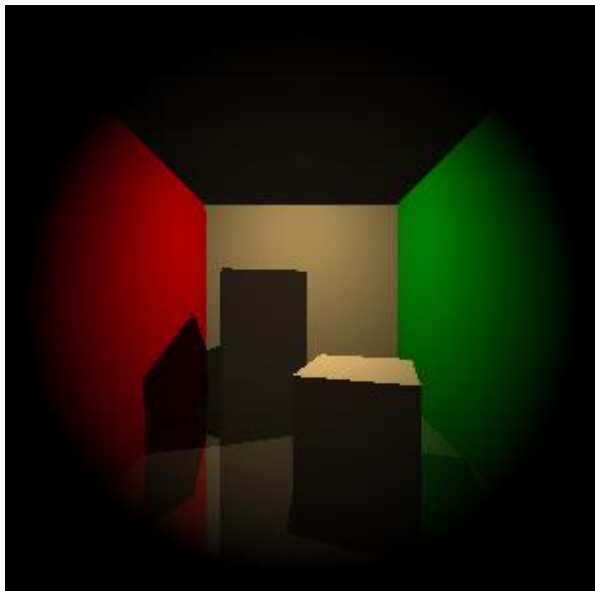
 distance = $\langle x, y \rangle - \text{center}$; [0, 1]

 vignette_factor = $1 - \text{distance}$; [0, 1]

 image[x,y] = lerp(image[x,y], fade_color, vignette_factor)

Vignette

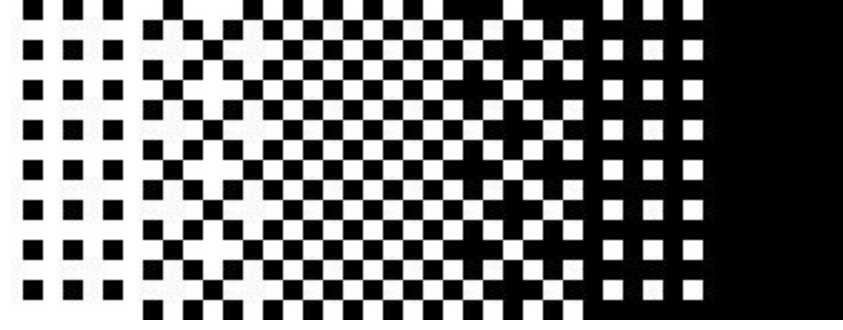




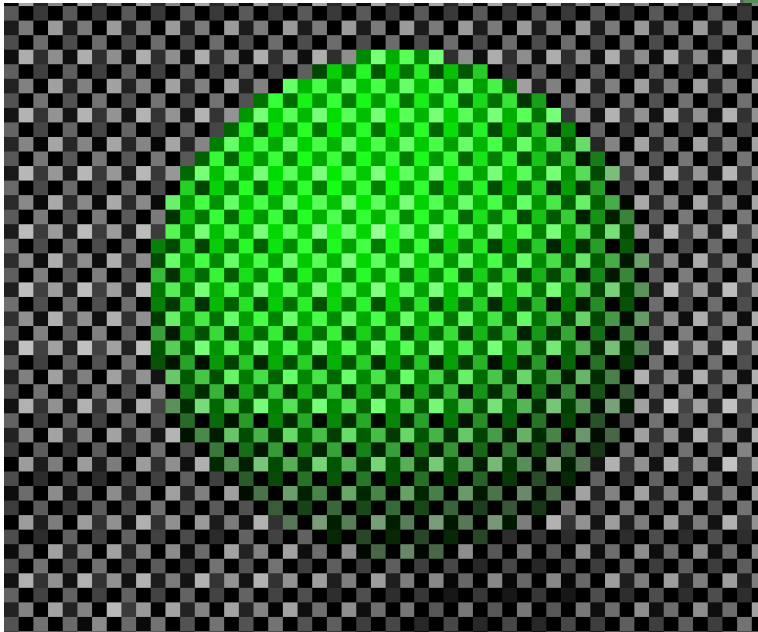
Dithering

- Goal: Reduce file size while preserving the "impression of the underlying feature"
- 2 models- Ordered and Error Diffusion
- 3 Steps:
 - 1) Select color palette
 - 2) Adjust pixel(s) by some amount
 - 3) Quantization

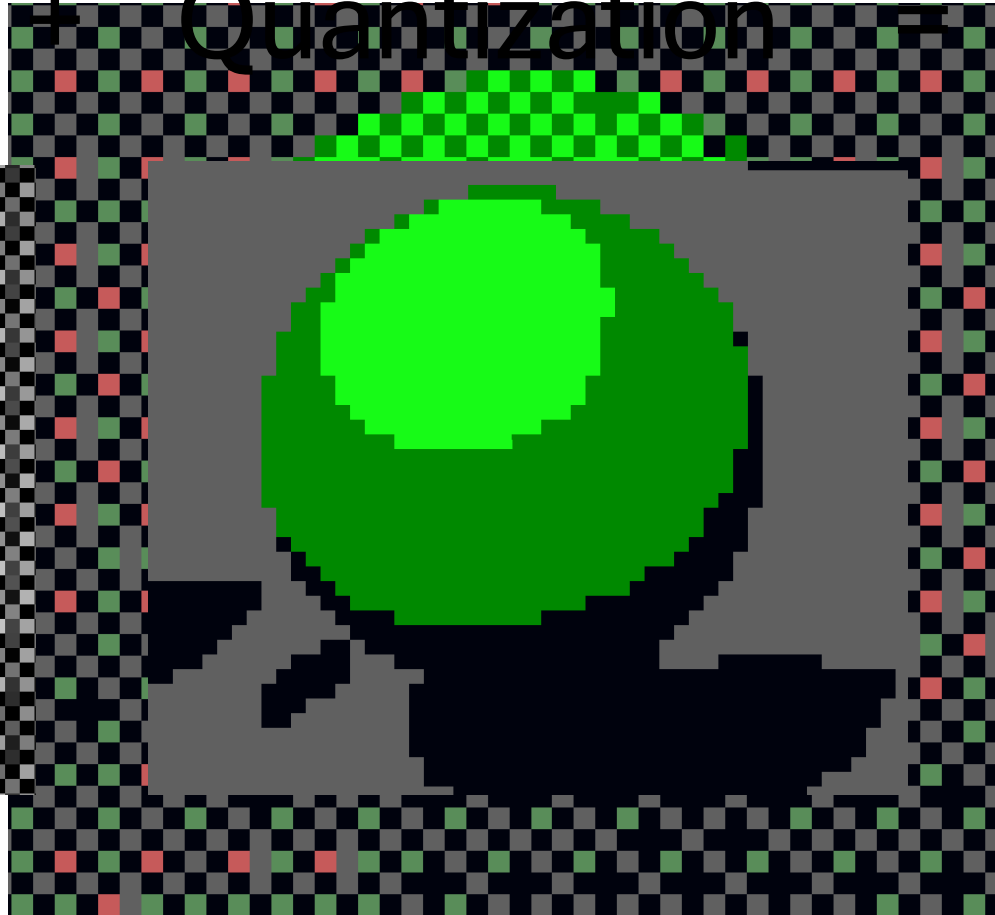
Ordered (Bayerian) Dithering



Threshold Map



Quantization



Output

Floyd-Steinberg Dithering (Error Diffusion)

for x in image; for y in image do

 pixel = image[x,y]

 quantized_color = closestColor(pixel)

 image[x,y] = quantized_color

 quantization_error = pixel – quantized_color

 image[x+1,y] += error * 7/16

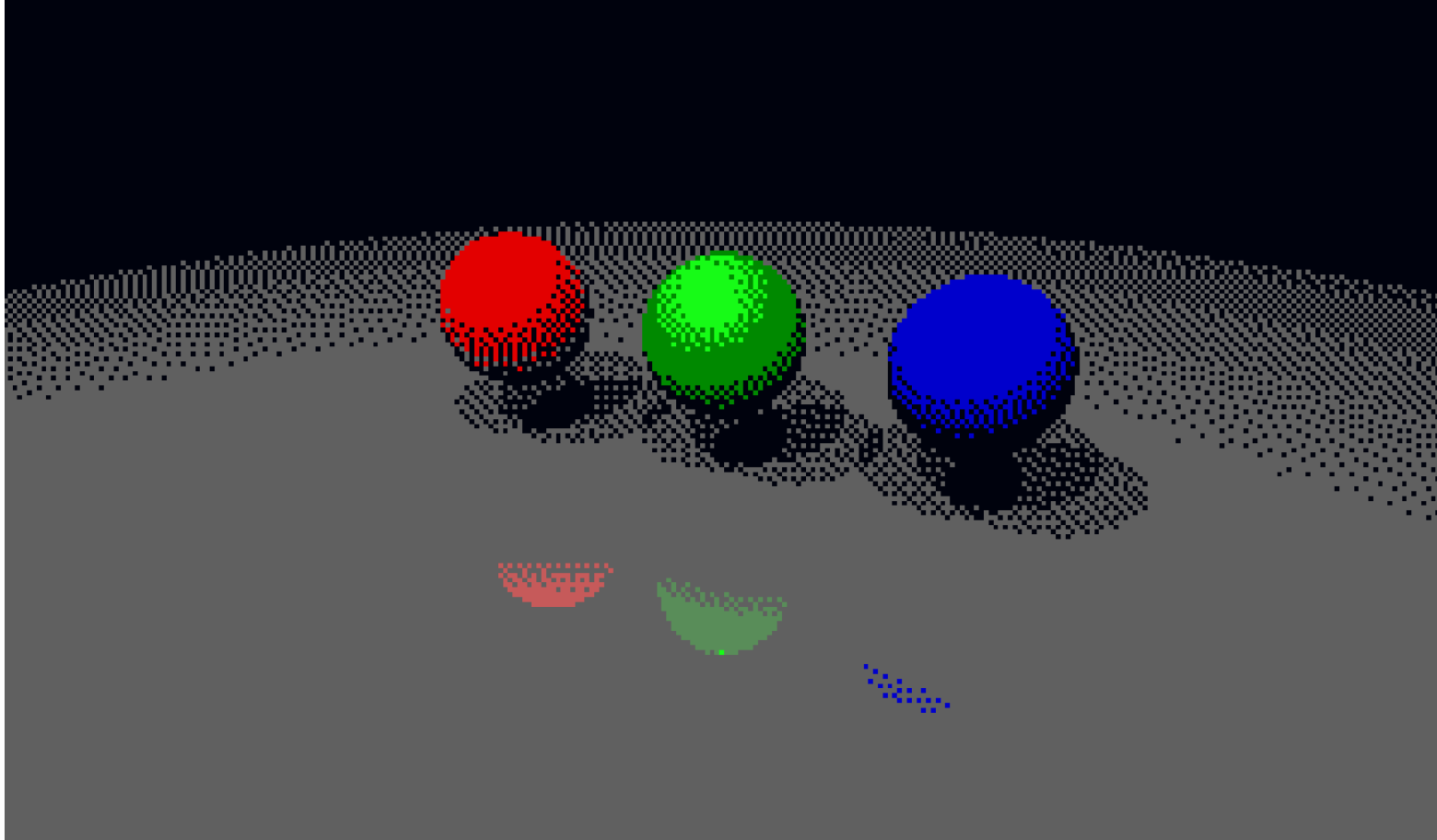
 image[x-1, y+1] += error * 3/16

 image[x, y+1] += error * 5/16

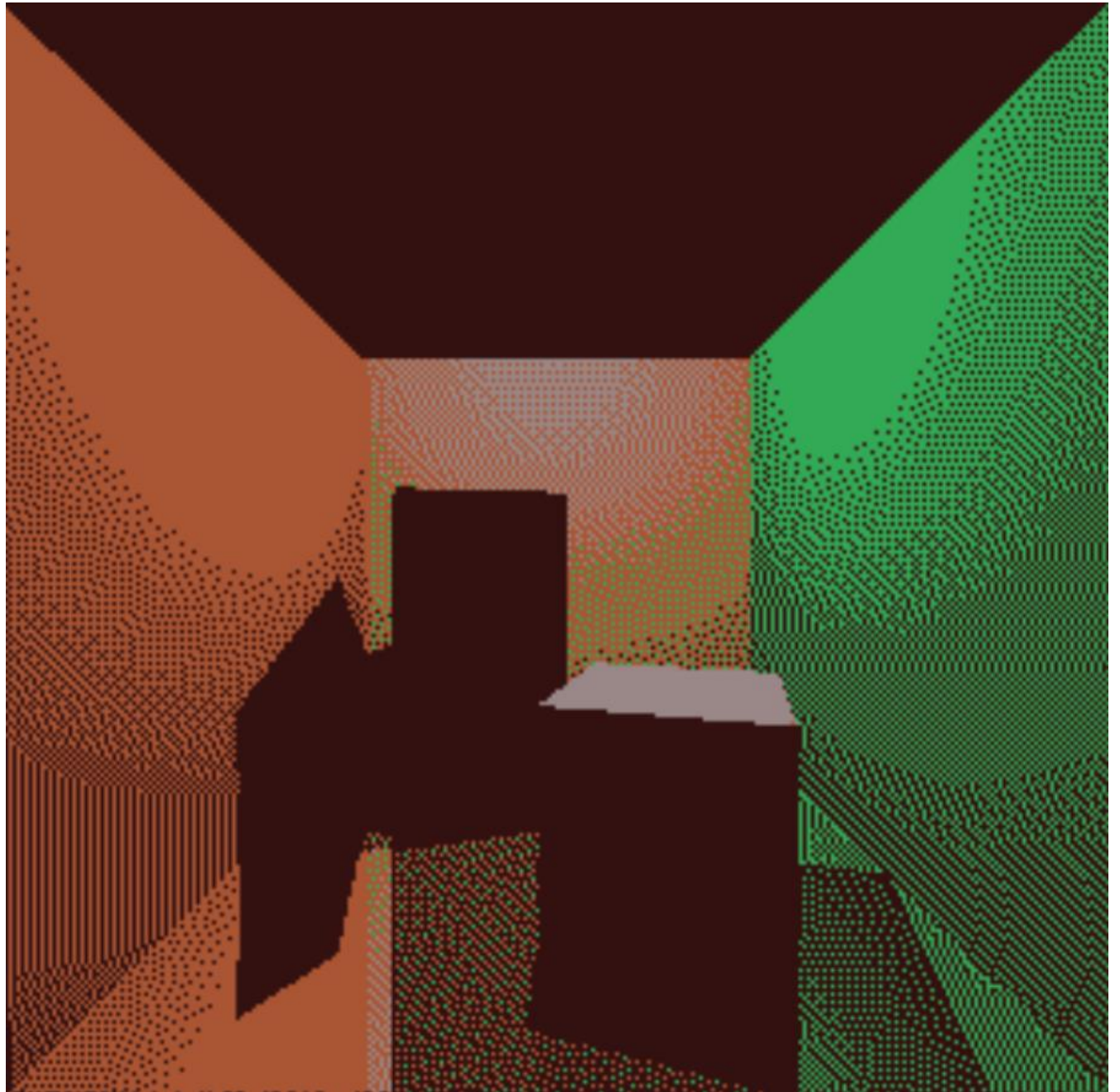
 image[x+1, y+1] += error * 1/16

$$\begin{bmatrix} & & * & \frac{7}{16} & \cdots \\ \cdots & \frac{3}{16} & \frac{5}{16} & \frac{1}{16} & \cdots \end{bmatrix}$$

Floyd-Steinberg Dithering (Error Diffusion)



Color Palettes in Dithering



Screen Space Ambient Occlusion (SSAOO)

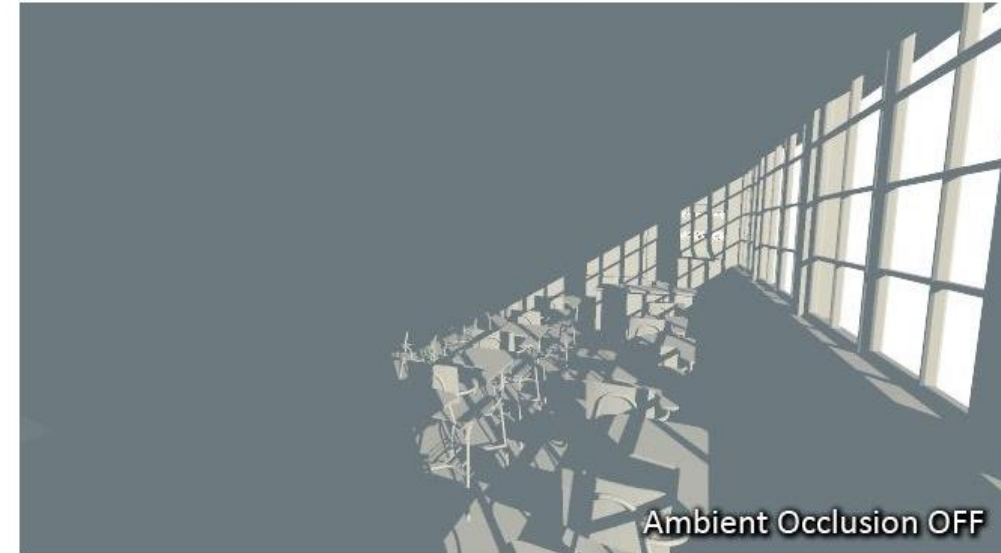
SSAO Goal: Simulate Ambient Occlusion in Post Process with less work

SSAO benefits:

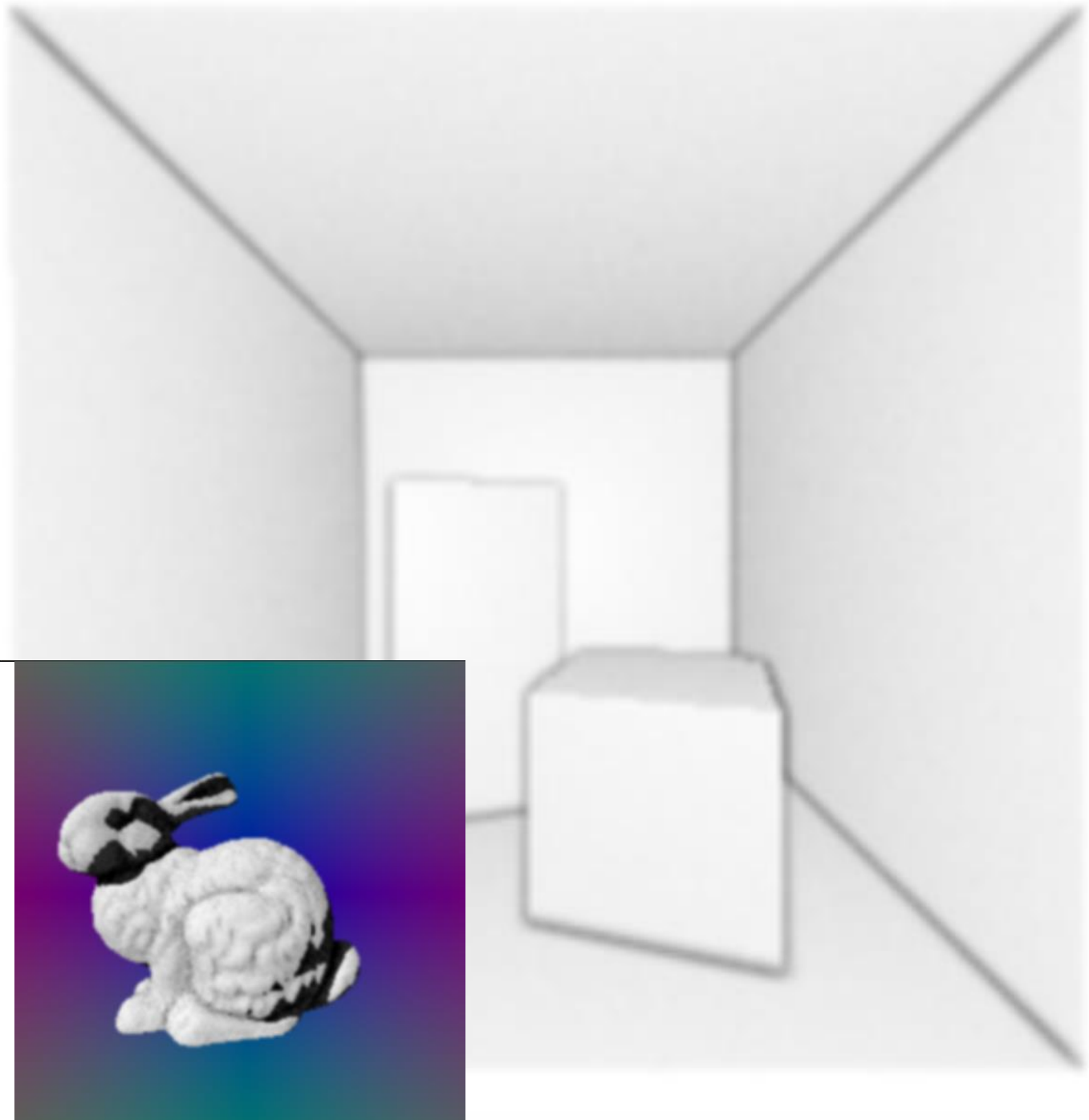
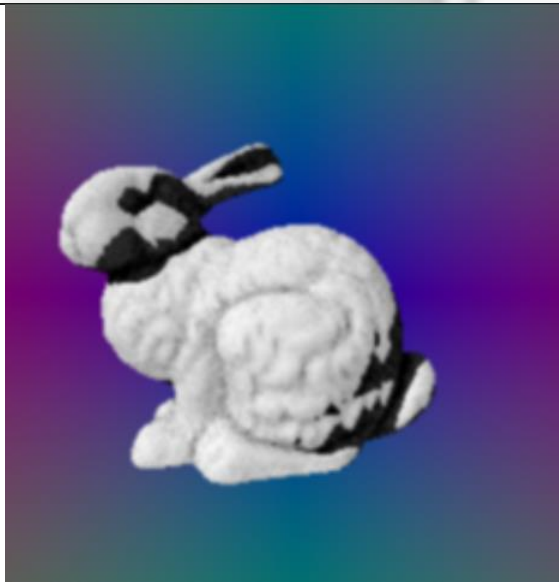
- Doesn't depend on complexity of scene just on size of depth buffer
- Can run just on the GPU no CPU compute required (our implementation is CPU though)

Ambient Occlusion Goal:

To make scenes appear more realistic with regard to ambient lighting.

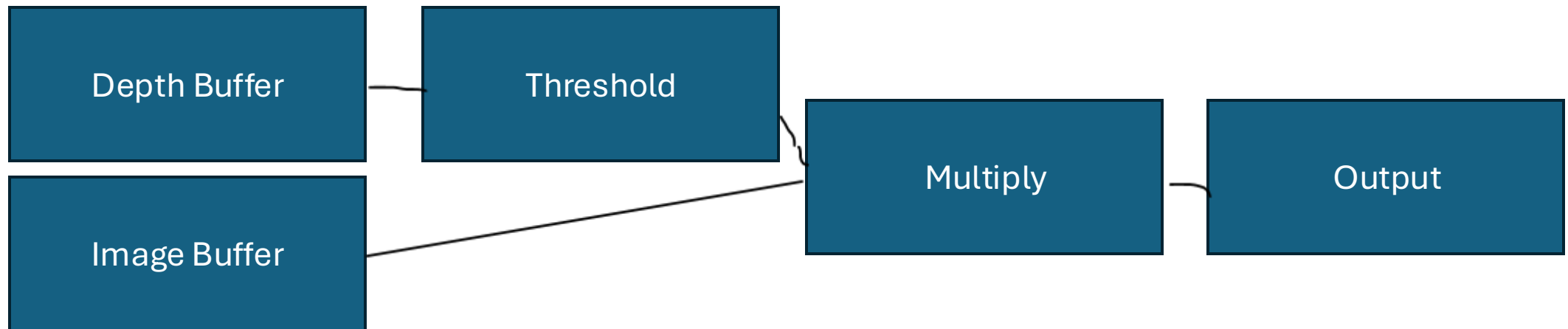


SSAOO continued

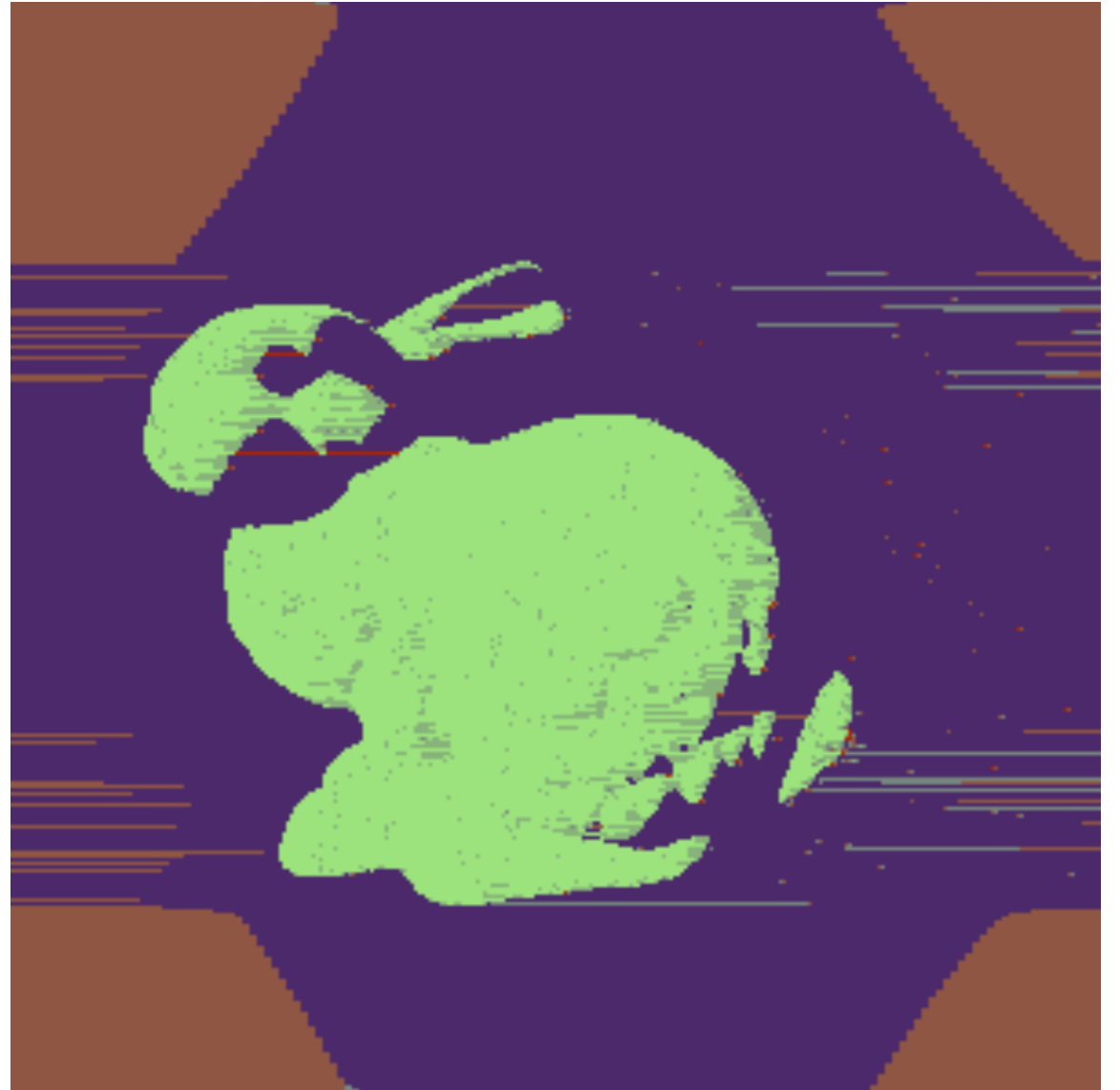
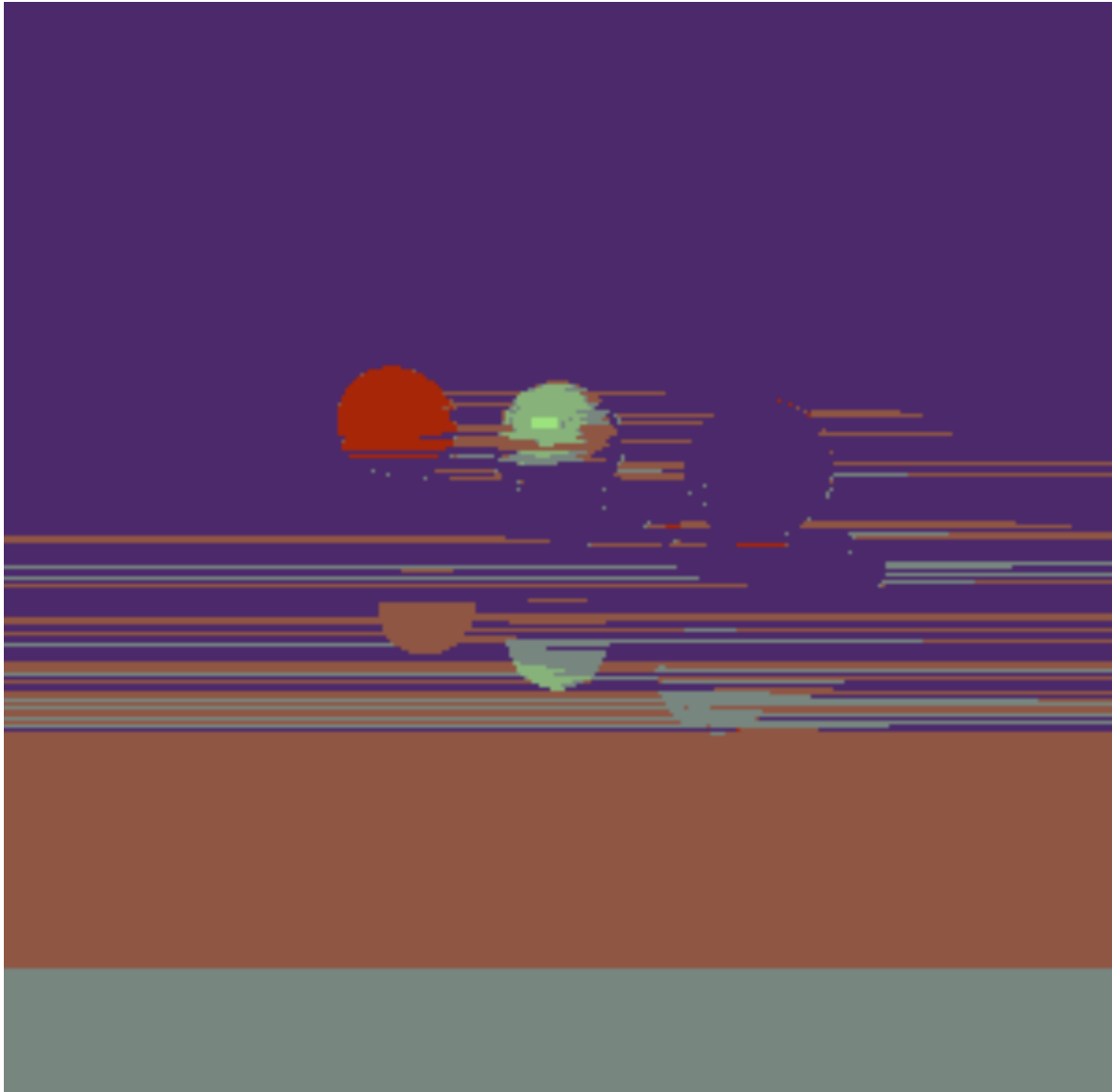


Pipeline, Blend Modes, Etc

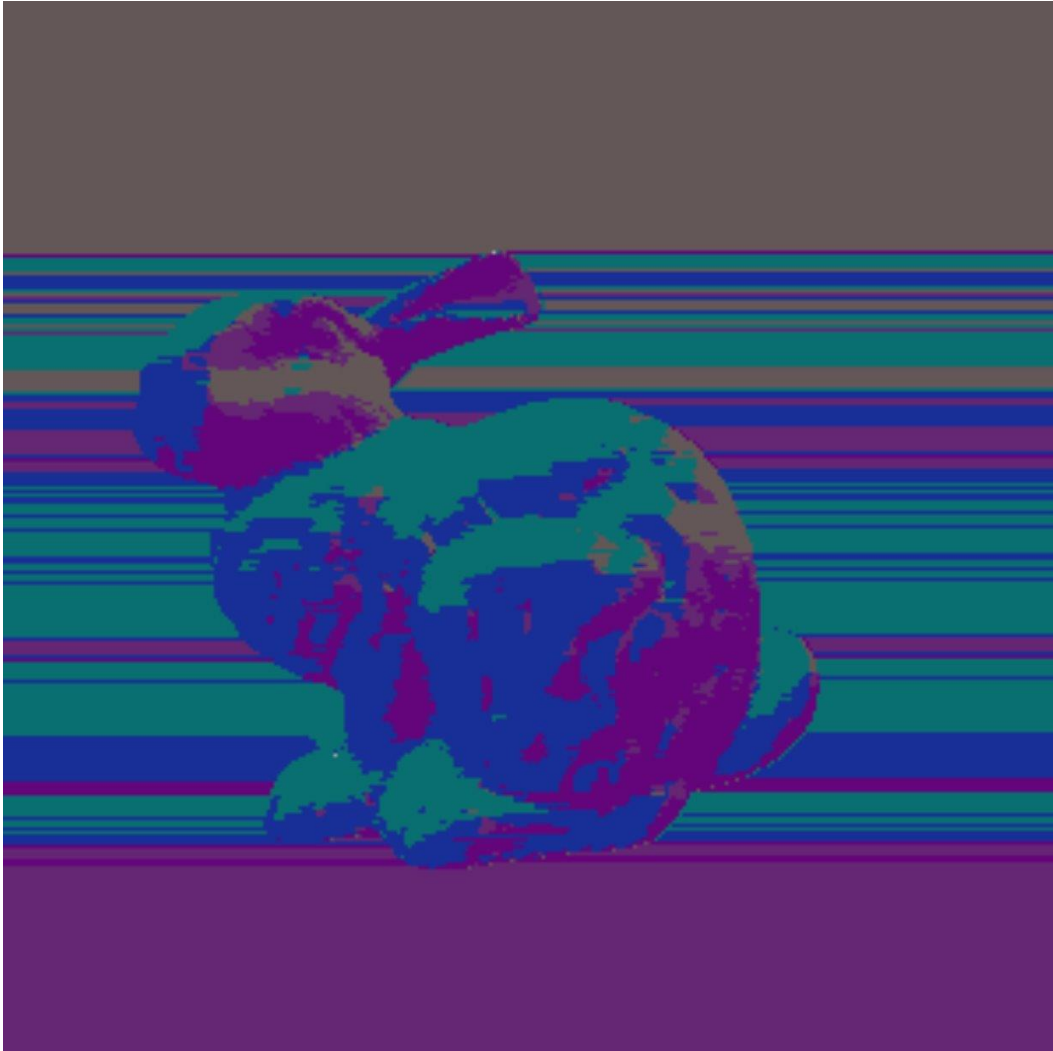
Inspired after Gstreamer Pipelining, Our postprocessing setup is equipped with lots of filters and blend modes for quickly and easily creating interesting combinations



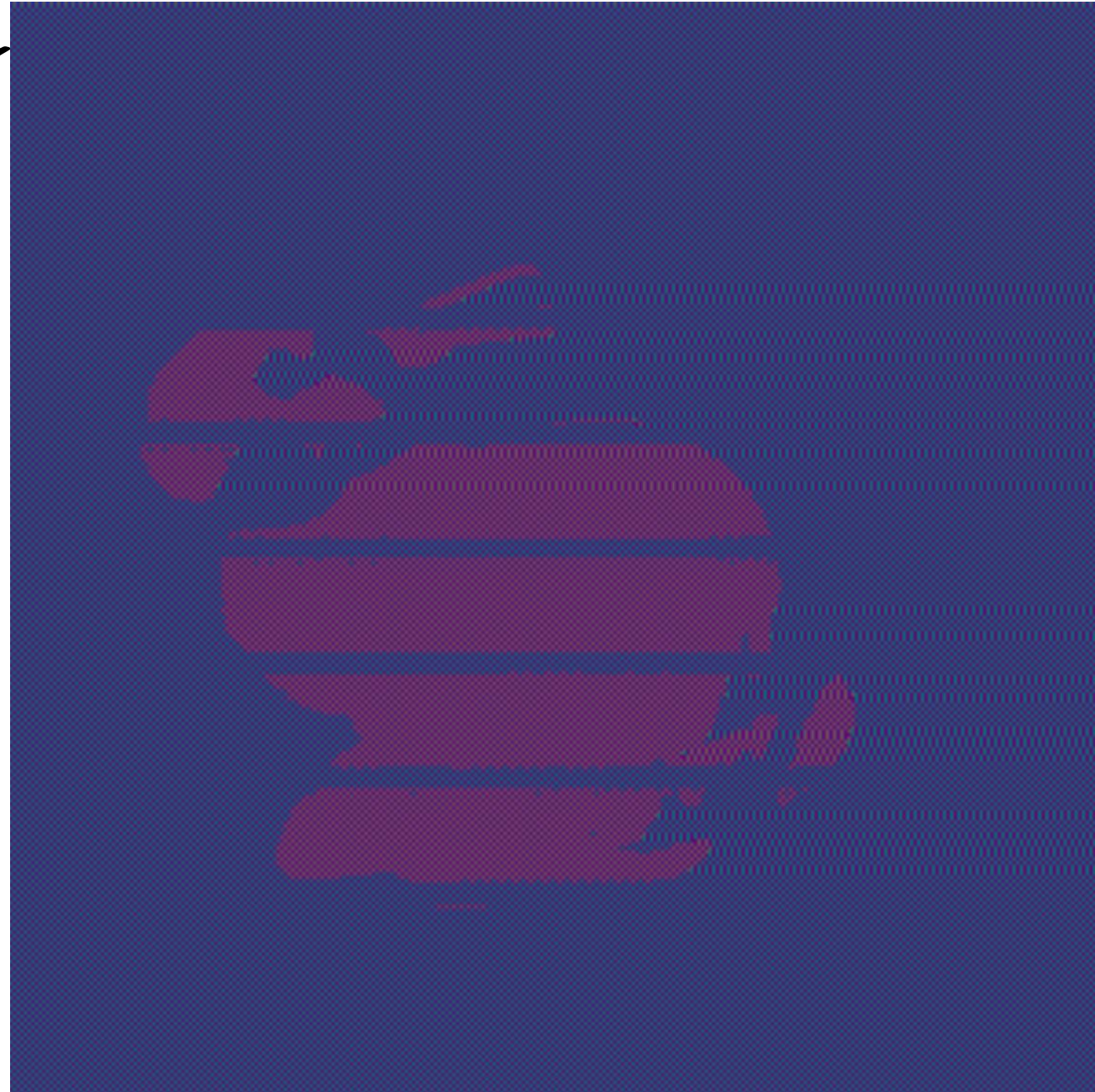
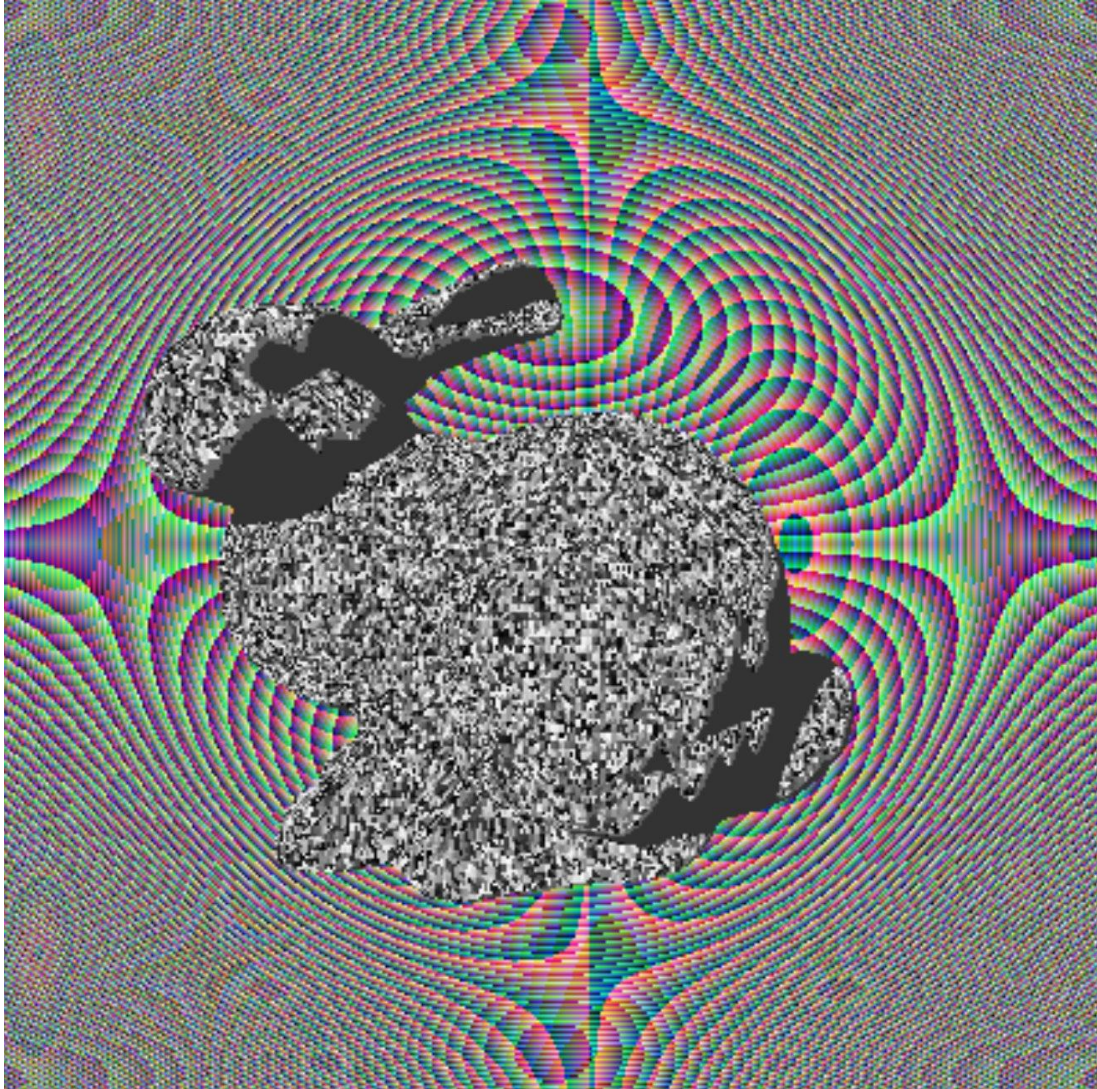
glitch art gallery



glitch art gallery



glitch art gallery



State of the Art

Done on the GPU

